Práctica 2. Implementación de una lista dinámica mediante plantillas y operadores en C++

Sesiones de prácticas: 2

Objetivos

Implementar y utilizar la clase `ListaDEnlazada<T>` y su clase auxiliar de tipo iterador ListaDEnlazada<T>::`Iterador` utilizando **patrones de clase y excepciones**. Programa de prueba para comprobar su correcto funcionamiento.

Descripción de la EEDD

Implementar la clase `ListaDEnlazada<T>` para que tenga toda la funcionalidad de una lista doblemente enlazada en memoria dinámica descrita en la Lección 7, utilizando patrones de clase y excepciones. Los métodos a implementar serán los siguientes:

- Constructor por defecto `ListaDEnlazada<T>()`
- Constructor copia `ListaDEnlazada<T>(const ListaDEnlazada<T>& origen)`
- Operador de asignación (=)
- Obtener los elementos situados en los extremos de la lista: `T& inicio()` y `T& Fin()`
- Obtener un objeto iterador para iterar sobre una lista bidireccionalmente: `ListaDEnlazada<T>::Iterador iterador ()` e implementar la funcionalidad completa del iterador.
- Insertar por ambos extremos de la lista, `void insertaInicio(T& dato)` y `void insertaFin(T& dato)`
- Insertar un dato en la posición anterior apuntada por un iterador: `void inserta(Iterador &i, T &dato)`
- Borrar el elemento situado en cualquiera de los extremos de la lista, `void borraInicio()` y `void borraFinal()`
- Borrar el elemento referenciado por un iterador: `void borra(Iterador &i)`
- `tam():entero`, que devuelve de forma eficiente el número de elementos de la lista.
- `concatena(const ListaDEnlazada<T> &l):ListaDEnlazada<T>`, que devuelve una nueva lista con la concatenación de la lista actual (this) y la proporcionada por parámetro. Sobrecargar también el *operador+* para realizar la misma funcionalidad.
- El destructor correspondiente.

Proyecto de prueba: Gestión de control de parques

ControlParques es el sistema que utiliza la Comunidad de Madrid para registrar los parques que tienen, así como las especies de árboles que contiene cada parque. En el siguiente UML podemos ver la clase *Especie* ya implementada en la Práctica 1, la nueva clase *Parque* que representa todos los parques de la Comunidad de Madrid. y la clase principal *ControlParques*. La clase *Parque* almacena, el código del parque (*cod_parque*) y el nombre del parque (*nbre_parque*). Cada parque contiene muchas especies representado mediante la relación *Parque::contains* (se implementará con un Vdinamico), y se añaden mediante la función *Parque::insertarEspecie()*. Además, la clase *Parque* permite realizar distintos tipos de búsquedas:

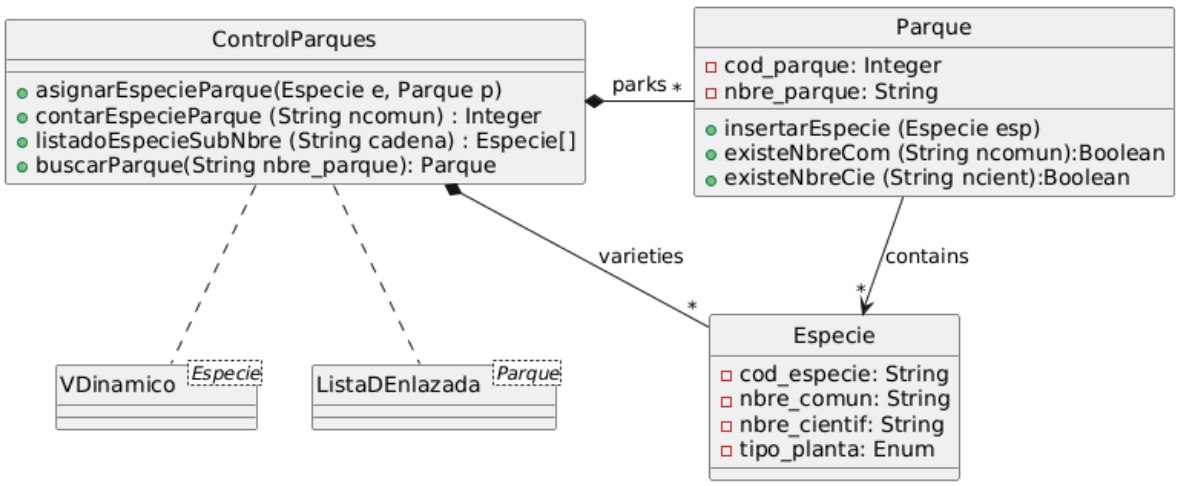
- *Parque::existeNbreCom ():* se encarga de buscar una Especie dado su nombre común y devuelve verdadero si la encuentra, en caso contrario, devolverá falso.
- *Parque::existeNbreCie():* se encarga de buscar una Especie a partir de su nombre científico, si la encuentra devuelve verdadero, en caso contrario devolverá falso.

La clase principal *ControlParques* representa el sistema de gestión de parques. Guarda las especies en un vector dinámico como en la Práctica 1 y los parques en una lista doblemente enlazada.

Por el momento, la funcionalidad de la clase *ControlParques* es básica, se va a encargar de hacer algunas búsquedas y enlazar las clases:

- *ControlParques::asignarEspecieParque():* se encarga de enlazar la especie con el parque que la contiene.
- *ControlParques::contarEspecieParque()* : se encarga de contar los parques que tienen una especie dando como parámetro su nombre común.
- *ControlParques::listadoEspeciesSubNbre():* devuelve aquellas especies que contengan en su nombre científico una subcadena dada como parámetro.
- *ControlParques::buscarParque():* se encarga de buscar un parque dado su nombre.

IMPORTANTE: no devolváis objetos copia



Programa de prueba I: probar la lista doblemente enlazada con enteros

- Implementar la EEDD *ListaDEnlazada<T>* y el Iterador<T> con la funcionalidad señalada arriba y de acuerdo con la especificación de la Lección 7.

Probar la robustez de la lista implementando la siguiente funcionalidad:

- Crear una lista de enteros inicialmente vacía.
- Insertar al final de la lista los valores crecientes desde 101 a 200. Mostrar la lista.
- Insertar por el comienzo de la lista los valores decrecientes desde 98 a 1. Mostrar la lista.
- Insertar el dato 100 delante del 101. Mostrar la lista.
- Insertar el dato 99 detrás del 98. Mostrar la lista.
- Borrar de la lista los 10 primeros y los 10 últimos datos. Mostrar la lista.
- Borrar de la lista todos los múltiplos de 10. Mostrar la lista.

Programa de prueba II: probar la funcionalidad de ControlParques

1. Leer el fichero "*arbolado-especies.csv"* para cargarlos en el vector dinámico *ControlParques::varieties* (similar a Pr1). Ordenarlo según su código de menor a mayor.
2. Adaptar el código de lectura del fichero anterior para leer el nuevo fichero *"parques.csv"*. Durante el proceso de lectura, los datos se insertarán en la lista doblemente enlazada en orden de lectura, *ControlParques::parks*. El identificador de cada parque es el *cod_parque*.
3. Enlazar de forma automatizada los parques (un total de 32) con las especies (un total de 4497) de la siguiente forma (*Parque::contains*):
   Si el tipo de planta de la especie es ARBUSTO o CONIFERA o PALMACEA o HERB_PERENNES o FRONDOSA entonces esa especie se asigna a los parques cuyo código, *cod_parque* es un valor par, en caso contrario (si es TAPIZANTE_TREPADORA, CACTACEA_SUCULENTA, SUBTROPICAL_TROPICAL, HERB_ANUAL, HERB_ANUALES) se asigna a aquellos parques cuyo *cod_parque* es impar.

Una vez que la estructura está inicializada realizar las siguientes acciones:

1. Busca si el parque con nombre "PARQUE PRINCESA LEONOR" tiene las especies con nombre común: "Eucalipto rojo" y "Arce de sierra".
2. Busca si el parque con nombre "PARQUE MADRID RIO" tiene las especies con nombre científico: "Hebe floribunda" y "Ananas comosa".
3. Buscar los parques con nombre "PARQUE JUAN CARLOS III" y "VIVERO DE MIGAS CALIENTES". Mostrar su código, en caso de no existir debe indicar que no existe.
4. Buscar y mostrar en pantalla aquellas especies que contienen la subcadena "rosa" en su nombre científico.

5. Contar cuántos parques contienen la especie cuyo nombre común es “Abeto”. Mostrar el número de parques.

Para los que trabajan en parejas:

1. Eliminar la especie con nombre común “Madroño”, con especial cuidado para que los parques que tenía esa especie ya no la enlacen, es decir, apunten a *nullptr*. Añadir a *ControlParques* la función que tenga dicha funcionalidad.

Estilo y requerimientos del código:

1. El código debe ser claro, tener un estilo definido y estar perfectamente indentado, para ello se pueden seguir algunos de los estilos preestablecidos para el lenguaje C++ (http://geosoft.no/development/cppstyle.html).

2. Deben comprobarse todas los posibles errores y situaciones de riesgo que puedan ocurrir (desbordamientos de memoria, parámetros con valores no válidos, etc.) y lanzar las excepciones correspondientes, siempre que tenga sentido. Leer el tutorial de excepciones disponible en el repositorio de la asignatura en docencia virtual.

3. Se valorará positivamente la calidad general del código: claridad, estilo, ausencia de redundancias, etc.